



Кафедра информационных систем
в химической технологии

Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА



ИНФОРМАЦИОННЫЕ СИСТЕМЫ В ХТ

Лекция 22. Контейнерная виртуализация

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

-  Контейнерная виртуализация
-  Docker
-  Управление образами и контейнерами
-  Сборка образов контейнеров
-  Контейнеризация приложений
-  Подключение контейнеров к сети
-  Долговременное хранение данных

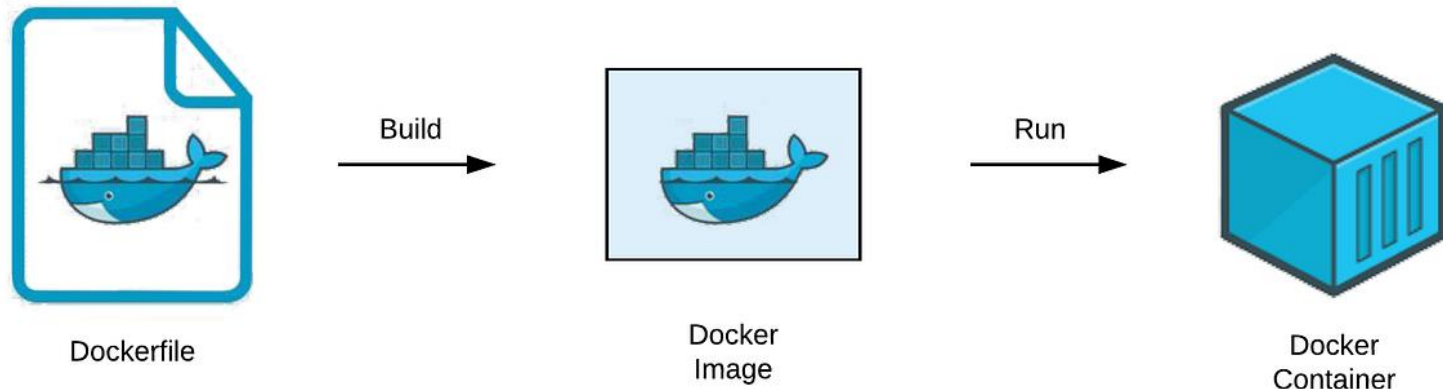
Контейнерная виртуализация

Контейнерная виртуализация позволяет поставлять приложение вместе с средой для запуска

- Упрощение тестирования и внедрения приложений
- Нет необходимости устанавливать зависимости и настраивать среду для запуска

Контейнер «легче» виртуальной машины

- Запуск и остановка за секунды
- Упрощается масштабирование и балансировка нагрузки

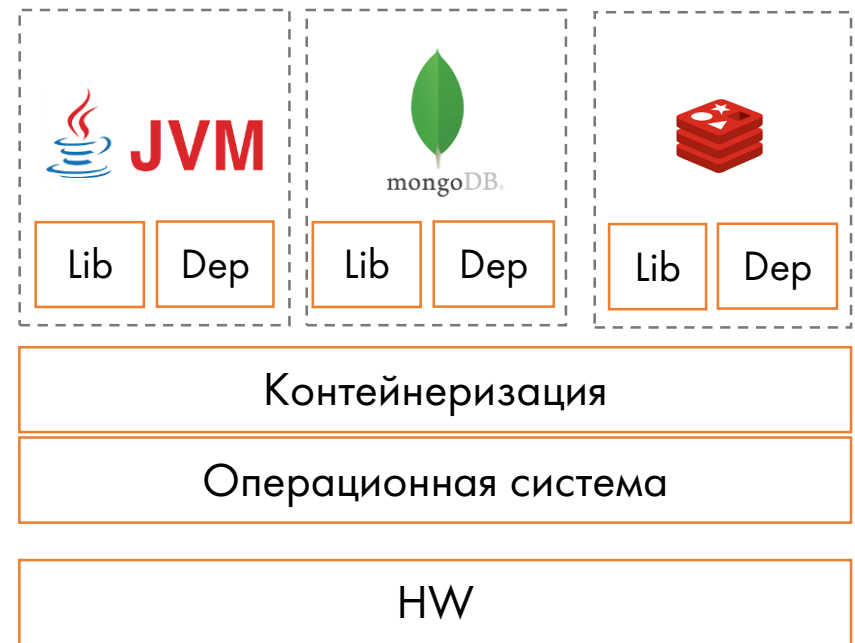
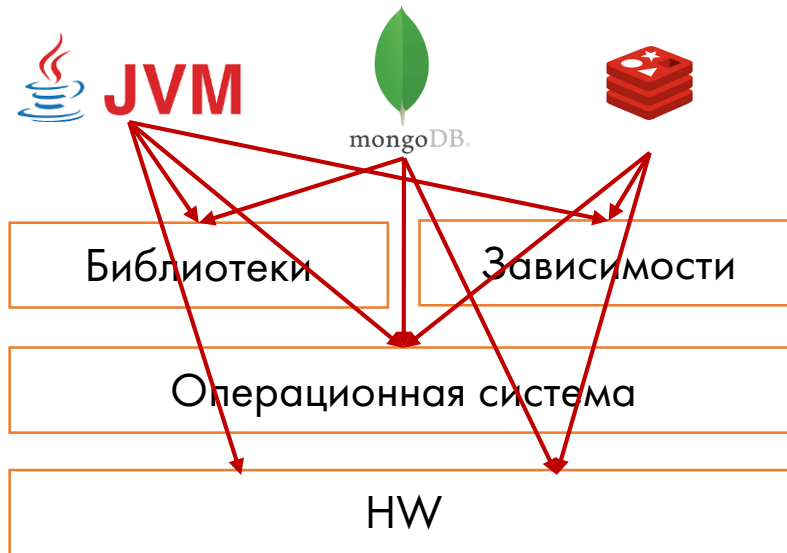


The Matrix from Hell

Проблема: различные компоненты приложения требуют различных сред или различных версий одной библиотеки

- При обновлении компонента – необходимо учитывать общие зависимости

Контейнерная виртуализация позволяет поставлять приложение вместе с средой для запуска



Архитектура контейнера

Контейнер работает напрямую на операционной системе

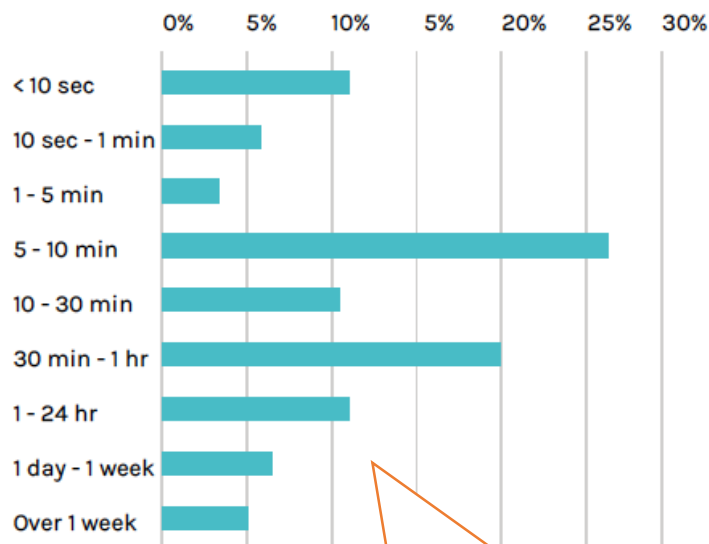
- Ограничение доступа с помощью функций ядра операционной системы
- Уменьшение служебной нагрузки на инфраструктуру



Концепция «Cattle vs pets»

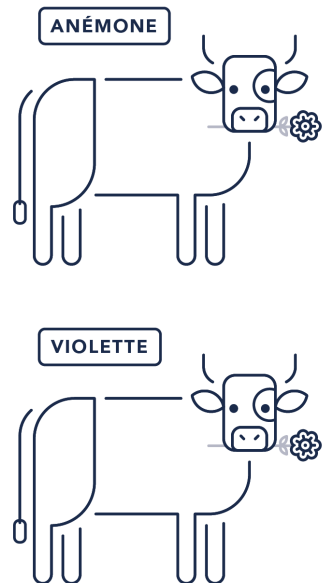
Контейнер может быть перезапущен в любой момент

- Обновление версии образа
- Ошибка в работе приложения
- Балансировка нагрузки между узлами

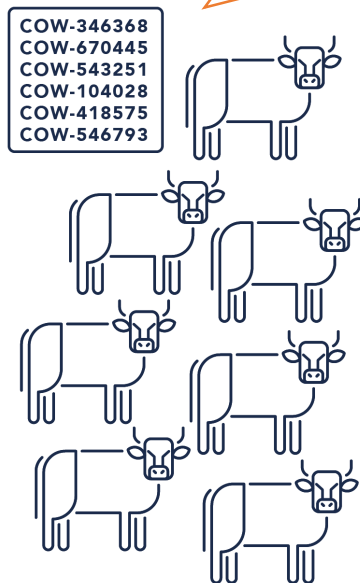


95% контейнеров «живут»
меньше недели

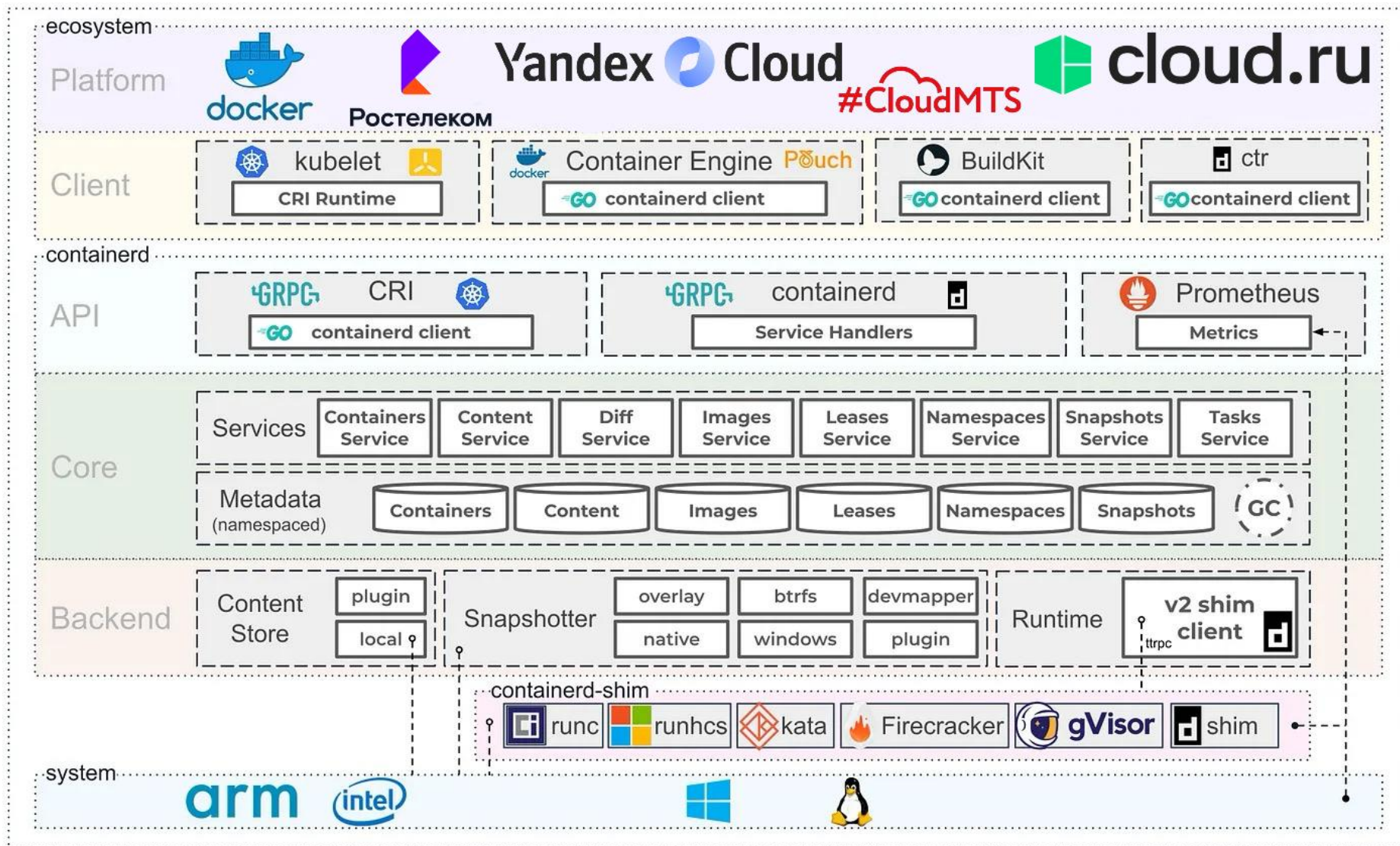
Контейнер – одна из многих
инстанций



VS

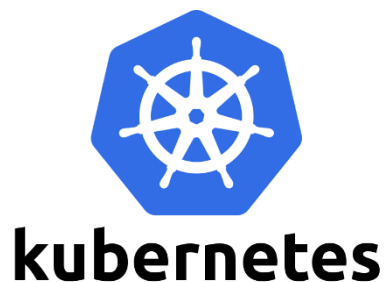


Контейнерная виртуализация. Экосистема (1 из 2)



Контейнерная виртуализация. Экосистема (2 из 2)

Наиболее популярные варианты внедрения Kubernetes



Наиболее частые приложения, запускаемые в контейнерах



Docker

Docker – наиболее популярная среда контейнерной виртуализации

- Разрабатывается с 2013 года, open-source
- Разработан на языке Go
- С 2015 года использует собственную библиотеку виртуализации libcontainer

Docker Community Edition и Docker Enterprise

- Репозиторий образов Docker Hub
- Платформа разработки Docker Desktop
- Управление группой контейнеров Docker Compose
- Встроенный оркестратор Docker Swarm
 - Поддерживается Kubernetes

Установка Docker в Linux

Установка Docker Desktop на Debian

● Настройка репозитория apt Docker:

```
# Add Docker's official GPG key:
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
echo \
  "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/debian \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
```

● Скачайте последнюю версию пакета Docker Desktop для Debian

Установка Docker в Linux

Установка пакета с помощью apt:

```
sudo apt-get update  
sudo apt-get install ./docker-desktop-<arch>.deb
```

- В конце процесса установки apt может показать ошибку, которую можно игнорировать:

```
N: Download is performed unsandboxed as root, as file '/home/user/Downloads/docker-desktop.deb' couldn't be accessed by user '_apt'. - pkgAcquire::Run (13: Permission denied)
```

Запуск Docker Desktop через терминал:

```
systemctl --user start docker-desktop
```

Установка Docker в Linux

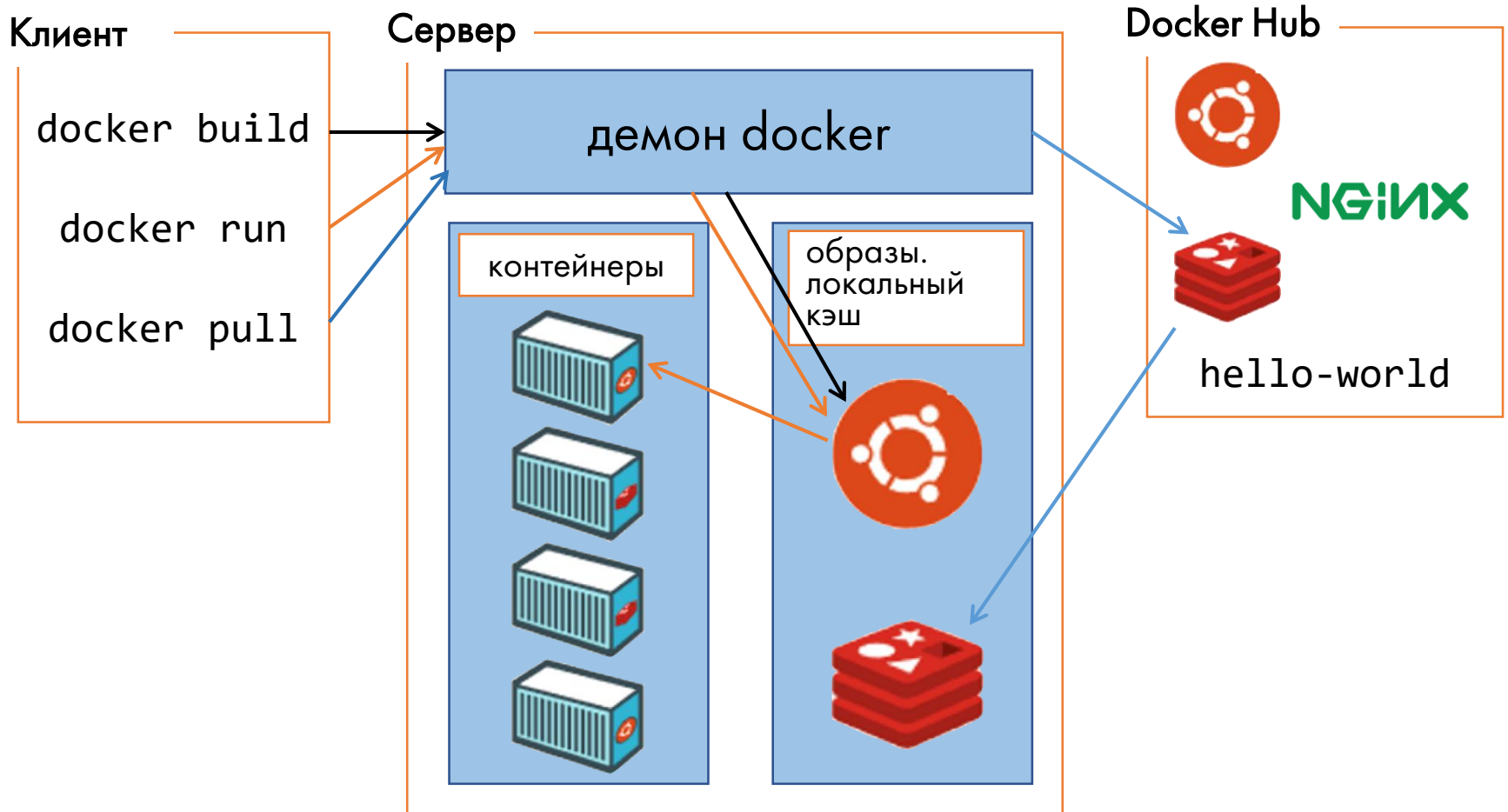
Проверка версий Docker

```
docker compose version
# Docker Compose version v2.17.3

docker --version
# Docker version 23.0.5, build bc4487a

docker version
# Client: Docker Engine - Community
# Cloud integration: v1.0.31
# Version:          23.0.5
# API version:      1.42
```

Архитектура Docker

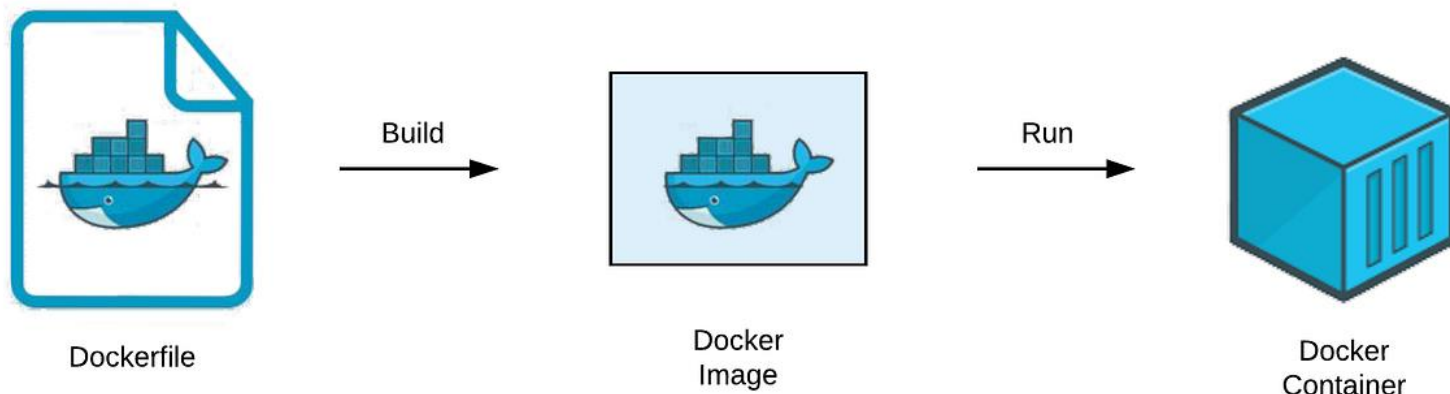


Образ и контейнер

Файл сборки образа содержит инструкции формирования и настройки образа, часто на основе другого образа

Образ содержит всю информацию для запуска контейнера в любой среде

Контейнер – запущенная инстанция образа



Контейнер

Контейнер – это группа процессов, изолированная от основной операционной системы и других групп

- Администратор может контролировать уровень изоляции сети, данных и пр. ресурсов

Контейнер – это запущенная инстанция образа

- Создание, запуск, остановка, перемещение, удаление и пр. реализовано с помощью Docker API и CLI

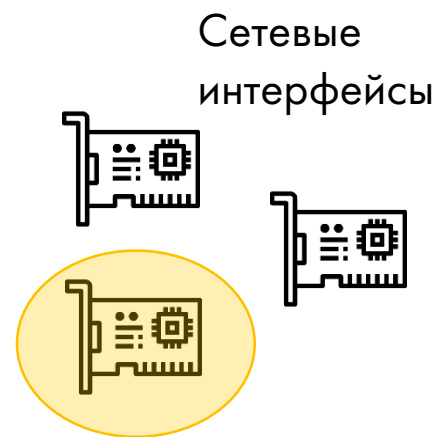
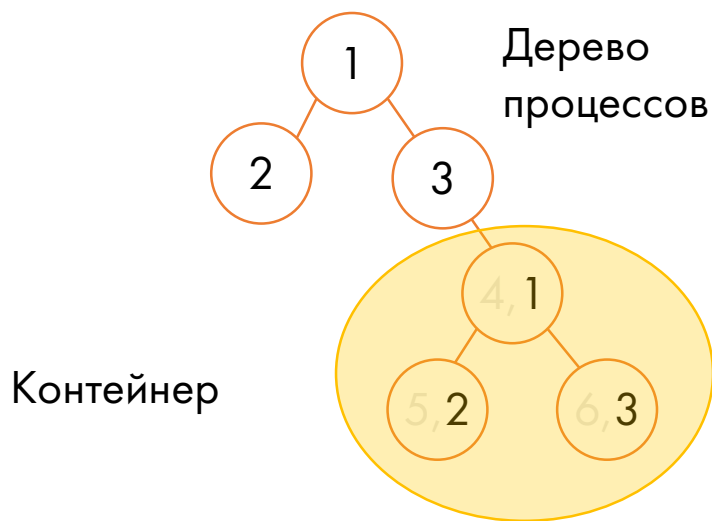
Контейнер независим

- Контейнер определяется образом и опциями запуска
- При удалении контейнера все изменения, не сохраненные на постоянное хранилище, удаляются

Linux. Механизмы контейнеризации (1 из 3)

Пространство имен (namespace) — функция ядра Linux, позволяющая изолировать и виртуализовать системные ресурсы групп процессов

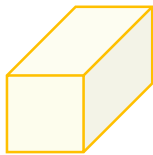
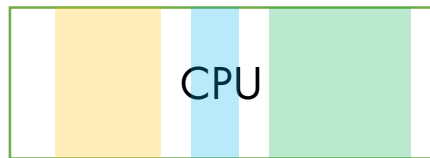
- Идентификаторы процессов
- Идентификаторы пользователей
- Файловые системы
- Доступ к сетевым интерфейсам и пр.



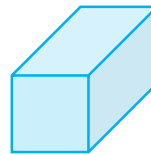
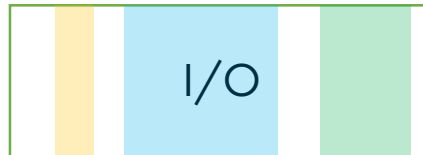
Linux. Механизмы контейнеризации (2 из 3)

Контрольная группа (cgroups) — механизм изоляции группы процессов в Linux

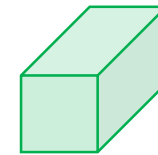
- Ограничение вычислительных ресурсов и памяти
- Ограничение сети и ресурсов ввода-вывода



Контейнер 1. App
A



Контейнер 2.
App B



Контейнер 3.
App C

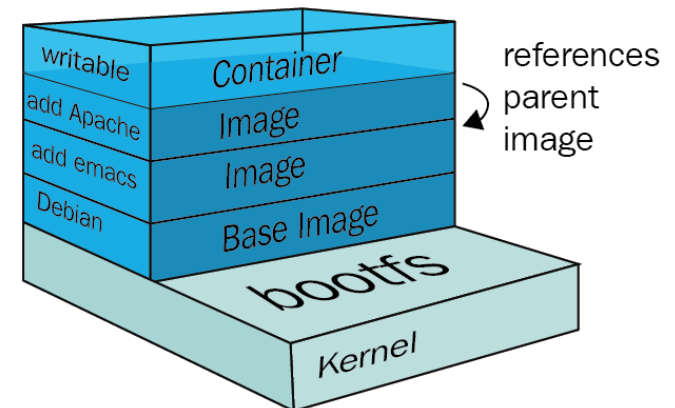
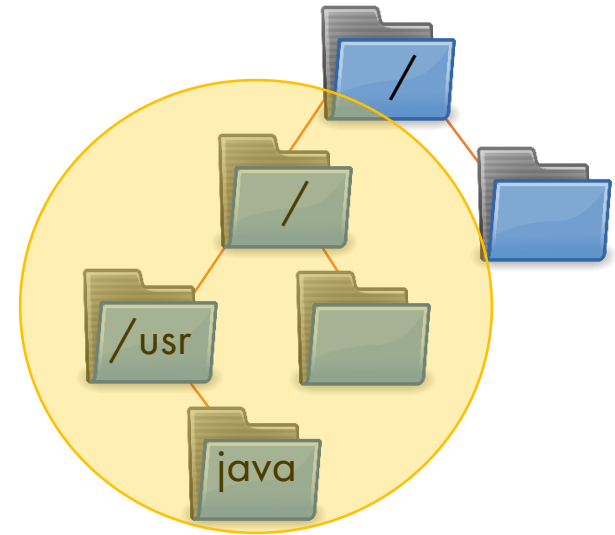
Linux. Механизмы контейнеризации (3 из 3)

Файловая система контейнера
изолирована от операционной системы

- Создается каталог rootfs
- Изоляция с помощью chroot

Файловая система контейнера состоит
из слоев

- Для записи доступен только верхний слой контейнера
- Слои «накладываются» друг на друга
- Слой может одновременно использоваться группой контейнеров



Образ

Образ – шаблон для запуска контейнера

- Содержит необходимые файлы для работы приложения
- Содержит описание инструкций для настройки и запуска приложения
- Часто образ зависит от других образов, например образа операционной системы
- Образ не изменяется при работе контейнеров

Dockerfile

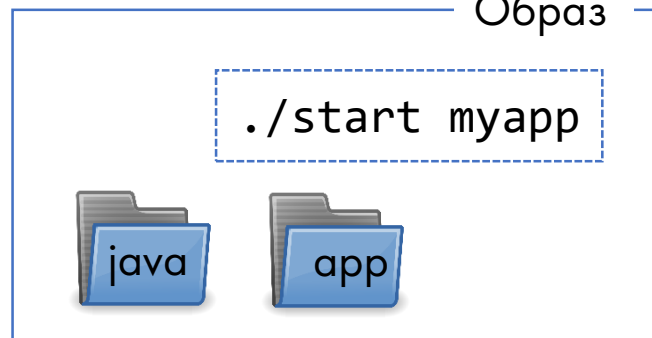
```
FROM ubuntu:15.04

RUN apt-get update && \
    apt-get install -y \
    openjdk-7-jre && \
    ...
    apt-get clean

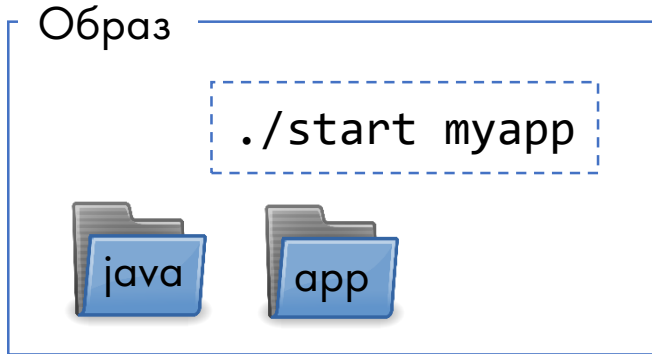
CMD ["start", "myapp"]
```



Образ



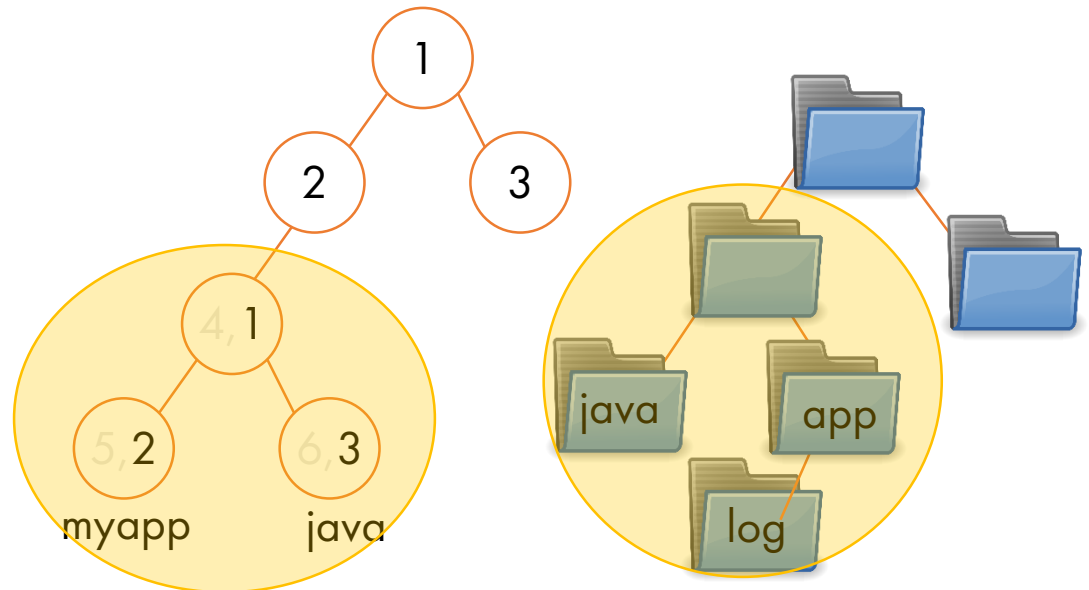
Запуск контейнера



Операционная система



`$ docker run myapp:latest`



Запуск контейнера HelloWorld

Создание и запуск контейнера

- Загружается образ из репозитория
- Формируется контейнер
- Запускается контейнер
- Транслируется вывод контейнера на терминал

```
$ docker run hello-world
```

```
Unable to find image 'hello-world:latest' locally
```

```
latest: Pulling from library/hello-world
```

```
ca4f61b1923c: Pull complete
```

```
Digest:
```

```
sha256:ca0eeb6fb05351dfc8759c20733c91def84cb8007aa89a5bf606bc8b315b9fc7
```

```
Status: Downloaded newer image for hello-world:latest
```

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

```
...
```

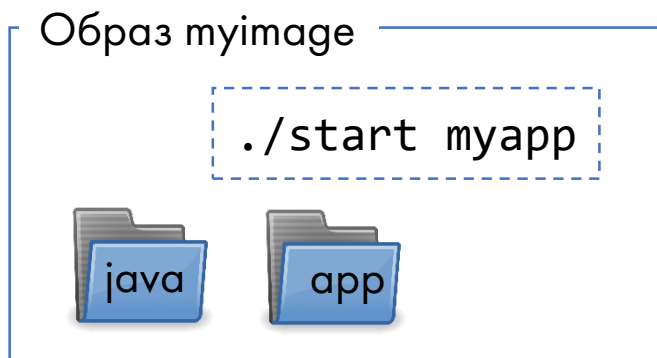
```
For more examples and ideas, visit:
```

```
https://docs.docker.com/get-started/
```

Создание и запуск контейнера

Команда, исполняемая при запуске образа, может быть изменена

- Исполняемый файл должен находиться внутри контейнера



```
$ docker run myimage myapp --diag
```

Создание контейнера без запуска

```
$ docker create myimage  
9aa88c08f319cd1e4515c3c46b0de7cc9aa75e878357b1e96f91e2c773029f03
```

Запуск остановленного контейнера

```
$ docker start 9aa88c
```


Просмотр контейнеров

Просмотр запущенных контейнеров

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
4d6fa6b8744e	busybox	"sleep 60"	3 seconds ago	Up 2 seconds

Просмотр всех контейнеров

● В том числе остановленных

```
$ docker ps --all
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
4d6fa6b8744e	busybox	"sleep 60"	11 seconds ago	Up 10 seconds
6dce868967e6	busybox	"sh"	39 seconds ago	Exited (0) 38 seconds
66a6530db601	hello-world	"/hello"	18 minutes ago	Exited (0) 15 minutes

или

```
$ docker ps -a
```

Остановка контейнера

Контейнер останавливается автоматически при завершении команды

- Можно остановить контейнер командами

```
$ docker stop 4d6fa6b8744e
```

```
$ docker kill 4d6fa6b8744e
```

Перезапуск остановленного контейнера

- Файлы, созданные во время предыдущего запуска, сохраняются

```
$ docker start 4d6fa6b8744e
```

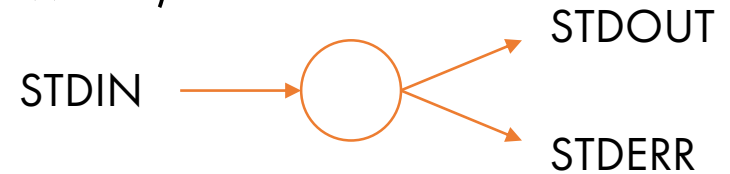
Очистка остановленных контейнеров

```
$ docker system prune
WARNING! This will remove:
- all stopped containers
- all networks not used by at least one container
- all dangling images
- all dangling build cache
Are you sure you want to continue? [y/N]
```

Использование терминала в контейнере

Процесс в Linux имеет стандартные потоки данных:

- STDIN - стандартный поток ввода (клавиатура)
- STDOUT - стандартный поток вывода (терминал)
- STDERR - стандартный поток ошибок (терминал)



Ассоциация потоков контейнера с терминалом

- `-i` ассоциировать поток ввода терминала с потоком ввода контейнера
- `-t` позволяет передавать в контейнер настройки терминала
- `-a` ассоциировать поток вывода терминала с потоком вывода контейнера

```
$ docker run -it busybox bash
```

```
$ docker start -a my_container
```

Отладка контейнера

Весь вывод контейнера хранится до момента его удаления

```
$ docker logs --timestamps efb62d0c1472
2023-10-03T07:42:10.804419776Z
2023-10-03T07:42:10.804506277Z Hello from Docker!
2023-10-03T07:42:10.804515877Z This message shows that your installation
appears to be working correctly.
2023-10-03T07:42:10.804523377Z
2023-10-03T07:42:10.804529977Z To generate this message, Docker took the
following steps:
2023-10-03T07:42:10.804536177Z 1. The Docker client contacted the
Docker daemon.
```

Список процессов, запущенных в контейнере

```
docker top <имя_или_ID_контейнера>
```

```
$ docker top f7ee980c6558
```

UID	PID	PPID	...	CMD
polkitd	35762	35741	...	mongod --bind_ip_all

Запуск дополнительных команд в контейнере

Запуск контейнера предполагает запуск основной программы

- Например, базы данных MongoDB

```
$ docker run mongo
Unable to find image 'mongo:latest' locally
latest: Pulling from library/mongo
707e32e9fc56: Pull complete
...
e596cadf5785: Pull complete
Digest:
sha256:26549961b7dd23ea104ad9ba0c30abec6689fda873070de9fdd2cad304af671d
Status: Downloaded newer image for mongo:latest
```

Запуск дополнительных программ для мониторинга, диагностики и пр. возможен командой `docker exec`

```
$ docker exec -it f77c11d435e5 bash
root@f77c11d435e5:/# ls
bin boot data dev docker-entrypoint-initdb.d etc home js-yaml.js/
lib lib32 lib64 libx32 media mnt opt proc root run sbin srv sys/
tmp usr var
```

Базовые команды Docker

Вывести список команд

```
$ docker  
$ docker container -help
```

Вывести информацию о Docker

```
$ docker --version  
$ docker version  
$ docker info
```

Запустить контейнер

```
$ docker run hello-world
```

Вывести список контейнеров

```
$ docker container ls  
$ docker ps  
$ docker ps -a
```

Вывести список образов

```
$ docker image ls
```

Docker Hub

Общедоступный репозиторий образов <https://hub.docker.com/>

+K?Sign InSign up

Filters

Products

- ☐ Images
- ☐ Extensions
- ☐ Plugins

Trusted Content

- ☐  Docker Official Image ⓘ
- ☐  Verified Publisher ⓘ
- ☐  Sponsored OSS ⓘ

Categories

- ☐ API Management
- ☐ Content Management System
- ☐ Data Science
- ☐ Databases & Storage
- ☐ Developer Tools
- ☐ Integration & Delivery
- ☐ Internet Of Things
- ☐ Languages & Frameworks

1 - 25 of 10,000 available results.

Suggested ▾

**postgres** 
Updated 6 days ago
The PostgreSQL object-relational database system provides reliability and data integrity.
DATABASES & STORAGE

1B+ · ☆10K+

Pulls: 40,393,529
Last week



[Learn more](#) 

**memcached** 
Updated 7 days ago
Free & open source, high-performance, distributed memory object caching system.
DATABASES & STORAGE

1B+ · ☆2.2K

Pulls: 121,110,649
Last week



[Learn more](#) 

**busybox** 
Updated 18 days ago
Busybox base image.
OPERATING SYSTEMS

1B+ · ☆3.3K

Pulls: 48,908,828
Last week



[Learn more](#) 

**alpine** 
Updated 21 hours ago

1B+ · ☆10K+

Pulls: 9,492,577
Last week



Файл сборки образа. Dockerfile

Dockerfile – текстовый файл, содержащий инструкции по сборке образа

- Образ может использовать готовый базовый образ
 - Например, образ операционной системы
- Каждая инструкция – отдельный уровень образа, который может быть использован повторно

```
$ cat Dockerfile
```

```
# Базовый образ  
FROM alpine
```

```
# Установка зависимостей  
RUN apk update
```

```
# Запуск команды / приложения  
CMD sh
```

Имя файла сборки всегда
Dockerfile (без расширения)

Linux Alpine – минимальная
сборка Linux, размер около 5 MB

Сборка образа

```
$ docker build -t test_image .
```

```
Sending build context to Docker daemon 7.168kB
```

```
Step 1/3 : FROM alpine:latest
```

```
latest: Pulling from library/alpine
```

```
e7c96db7181b: Pull complete
```

```
Digest:
```

```
sha256:769fddc7cc2f0a1c35abb2f91432e8beecf83916c421420e6a6da9f8975464b6
```

```
Status: Downloaded newer image for alpine:latest
```

```
---> 055936d39205
```

```
Step 2/3 : RUN apk update && apk upgrade
```

```
---> Running in d144ef3e9d3d
```

```
fetch http://dl-cdn.alpinelinux.org/alpine/v3.18/main/x86_64/APKINDEX.tar.gz
```

```
v3.18.0-1-g9b1e7d5a9c [http://dl-cdn.alpinelinux.org/alpine/v3.18/main]
```

```
v3.18.0-1-g9b1e7d5a9c [http://dl-cdn.alpinelinux.org/alpine/v3.18/community]
```

```
OK: 9770 distinct packages available
```

```
Removing intermediate container d144ef3e9d3d
```

```
---> c22e5a5ffadd
```

```
Step 3/3 : CMD ["bash"]
```

```
---> Running in 3f721614f2f7
```

```
Removing intermediate container 3f721614f2f7
```

```
---> 3afd859f74a5
```

```
Successfully built 3afd859f74a5
```

```
Successfully tagged test_image:latest
```

Базовый образ загружается из локального кеша или из репозитория

Результат выполнения каждого уровня сохраняется как образ для повторного использования

Тег позволяет обращаться к образу по имени и отслеживать версии образа

Инструкции Dockerfile. FROM и RUN

Инструкция FROM указывает на базовый образ операционной системы или приложения

- Первая инструкция в файле сборки
 - Может быть предварена только инструкцией ARG
- Выбор базового образа зависит от потребностей приложения

```
ARG VERSION=18.04  
FROM ubuntu:${VERSION}
```

Инструкция RUN позволяет настроить базовый образ перед запуском приложения

- Каждая инструкция – отдельный уровень образа
- Одна инструкция может содержать более одной команды с помощью оператора &&

```
RUN apt-get update && apt-get install -y x11vnc xvfb firefox  
RUN mkdir ~/.vnc
```

Инструкции Dockerfile. CMD

Инструкция CMD определяет команду, запускаемую при старте контейнера

- Может быть переопределена при запуске контейнера
- Файл сборки может содержать только одну инструкцию CMD
 - Если несколько – только последняя будет использована
- Предпочитаемый формат

```
CMD ["/usr/bin/wc", "--help"]
```

- ENTRYPOINT определяет команду, а CMD – опции и параметры

```
ENTRYPOINT ["/usr/bin/wc"]  
CMD ["--help"]
```

- Использование `/usr/bin/sh -c` для запуска команды

```
CMD wc --help
```

Инструкции Dockerfile. ENV и USER

Инструкция ENV позволяет настраивать образ под окружение и версию продукта

```
ENV PG_MAJOR 9.3
ENV PG_VERSION 9.3.4
RUN curl -SL http://example.com/postgres-$PG_VERSION.tar.xz |
    tar -xJC /usr/src/postgress && ...
ENV PATH="/usr/local/postgres-$PG_MAJOR/bin:$PATH"
```

Инструкция USER позволяет запускать программы из-под непривилегированного пользователя

```
FROM microsoft/windowsservercore
RUN net user /add patrick
USER patrick
RUN ["powershell", "-command", "Execute-MyCmdlet", "-param1 \"c:\\foo.txt\""]
```

Инструкции Dockerfile. Expose (1 из 2)

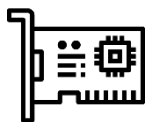
По умолчанию, процессы внутри контейнера имеют доступ к внешним ресурсам сети

- Например, для обновления или установки зависимостей приложения

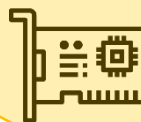
По умолчанию доступ внешних клиентов к процессам внутри контейнера запрещен

- Чтобы разрешить доступ к определенным портам, необходимо при запуске контейнера указать соответствие между портом операционной системы и портом контейнера

```
docker run -p 8080:80 web_server
```



localhost:8080



3f721614f2f7:80

Инструкции Dockerfile. Expose (2 из 2)

Dockerfile поддерживает инструкцию EXPOSE, описывающую какие порты необходимы для работы контейнера

- Публикация портов и задание соответствия портам операционной системы производится при запуске контейнера

```
EXPOSE 80/tcp  
EXPOSE 80/udp
```

```
docker run -P web_server
```

-P – автоматический выбор порта операционной системы для всех портов, описанных в инструкции EXPOSE

```
$ docker ps  
CONTAINER ID    IMAGE           ...    CREATED PORTS  
dc88d41dd031    web_server     ...    0.0.0.0:32768->80/tcp, 0.0.0.0:32768->80/udp  
  
$ docker port dc88d41dd031  
80/tcp -> 0.0.0.0:32768  
80/udp -> 0.0.0.0:32768
```

NB: Помните, что публикация портов не происходит автоматически

Передача информации в образ (1 из 2)

Контейнер изолирован от операционной системы на уровне файловой системы

- Для передачи информации в образ используются инструкции COPY и ADD

```
FROM alpine
RUN apk update
RUN apk add g++
RUN g++ main.cpp
CMD a.out
```



```
$ docker build .
...
Step 4/5 : RUN gcc main.cpp
---> Running in d6d55f7af90f
gcc: error: main.cpp: No such file or directory
gcc: fatal error: no input files
compilation terminated.
The command '/bin/sh -c gcc main.cpp' returned a
non-zero code: 1
```

```
FROM alpine
RUN apk update
RUN apk add g++
COPY ./ ./
RUN g++ main.cpp
CMD a.out
```



```
$ docker build .
...
Step 4/6 : COPY ./ ./
---> 0cc5c61940c1
Step 5/6 : RUN g++ main.c
---> Running in dc9656c4f606
```

Передача информации в образ (2 из 2)

Инструкция COPY копирует файлы или каталоги в образ

- Можно указать владельца файлов

```
COPY --chown=user:group ./dir_outside ./dir_inside/
```

- Файлы должны находиться внутри контекста сборки

```
COPY /tmp/abc ./
```

```
$ docker build .  
Step 4/6 : COPY /tmp/abc ./  
COPY failed: stat /var/lib/docker/tmp/docker-  
builder722303733/tmp/abc: no such file or directory
```

- Целевой каталог по умолчанию – корневой
 - Изменить можно с помощью инструкции WORKDIR
 - Если каталога не существует – он будет создан

```
WORKDIR /usr/myapp  
COPY ./main.cpp ./
```


Уровни образа

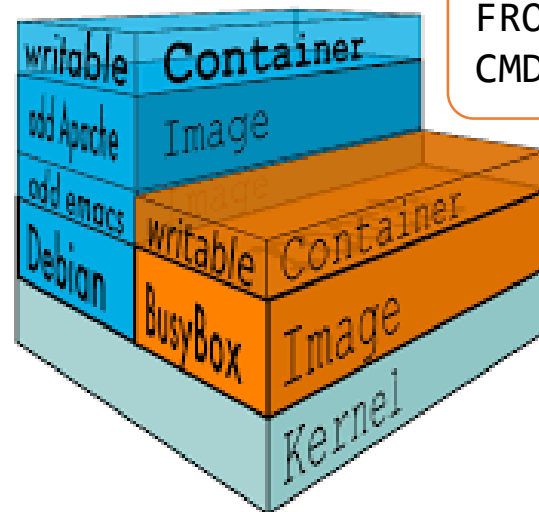
Файловая система контейнера состоит из слоев, созданных при сборке образа

- Уровни образа защищены от записи
- При изменении файла в контейнере он копируется в верхний слой – Copy-On-Write

Уровни могут быть использованы повторно в разных образах

- Уровень может быть подписан и защищен от изменений

```
FROM debian
RUN apt-get update &&
    apt-get install -y emacs
RUN apt-get install -y apache2
CMD ["apachectl", "-D", "FOREGROUND"]
```



```
FROM busybox
CMD sh
```

Docker предлагает гибкую настраиваемую сетевую модель

- Bridge – драйвер сети по умолчанию
 - Изоляция от операционной системы и внешних ресурсов
 - Взаимодействие контейнеров в сети
- Host – подходит для одиночных контейнеров
 - Нет изоляции сетевого стека контейнера от операционной системы
- Overlay – драйвер, который создает распределенную сеть между несколькими хостами Docker-демонов
 - Часто используется для кластера Docker Swarm
- MacVLAN – драйвер, имитирующий подключение физического интерфейса
- None – отсутствие сети внутри контейнера
- Можно устанавливать и использовать сетевые плагины для иных способов взаимодействия

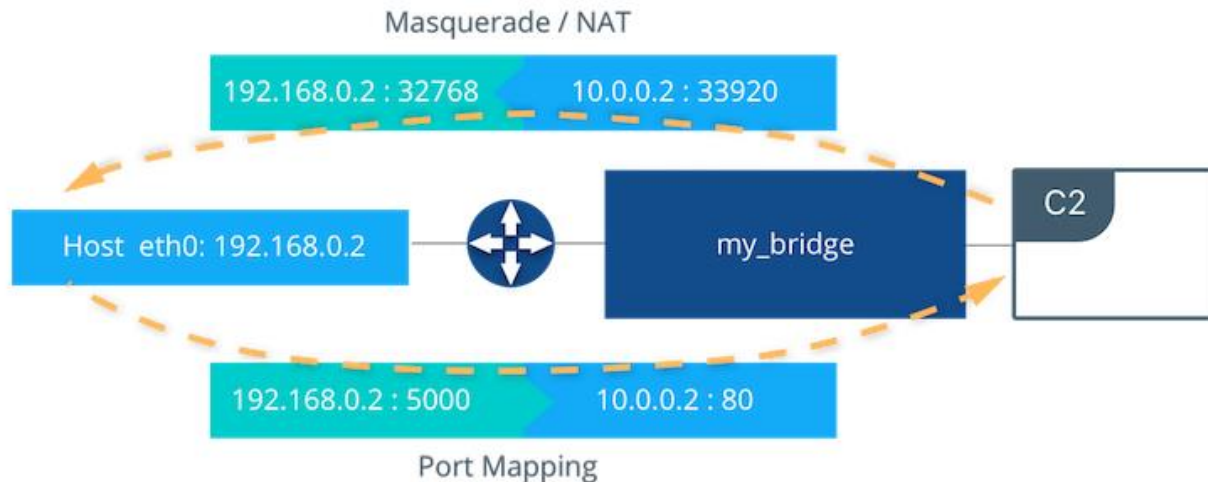
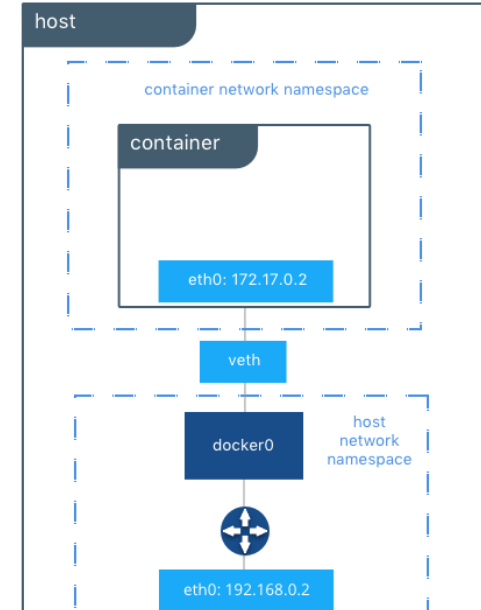
Тип сети Bridge

Тип сети bridge:

- Используется по умолчанию
- Можно создавать собственные сети

Изоляция и возможность взаимодействия:

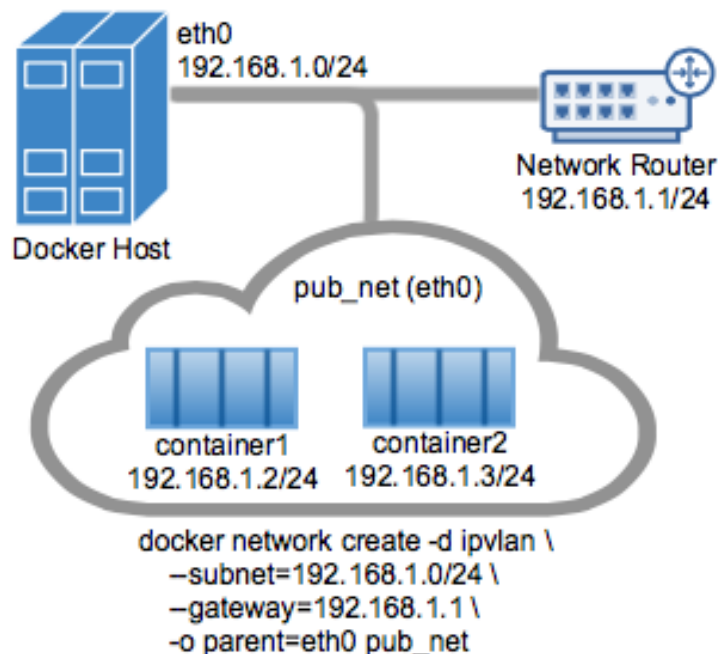
- Контейнеры изолированы полностью друг от друга внутри одной сети
- Все порты контейнера доступны другим контейнерам, находящимся в той же сети
- Автоматическое обнаружение контейнеров по имени (DNS)



MacVLAN и IPVLAN

Сеть MacVLAN и IPVLAN позволяет имитировать подключение физической машины к сети

- Для приложений сетевого мониторинга или при переносе приложения из виртуализации в контейнер
- Два подвида сети
 - Bridge – используется выделенный физический интерфейс
 - 802.1Q trunk bridge - позволяет настраивать сетевые под-интерфейсы



Хранение данных приложения (1 из 2)

Одно из основных свойств контейнера – эфемерность:

- то есть, контейнер в любой момент может быть удален и повторно развернут из образа.

По умолчанию данные, создаваемые приложением, хранятся внутри контейнера

- При остановке контейнера данные сохраняются
- При удалении контроллера данные удаляются (!)
- Данные недоступны извне контейнера

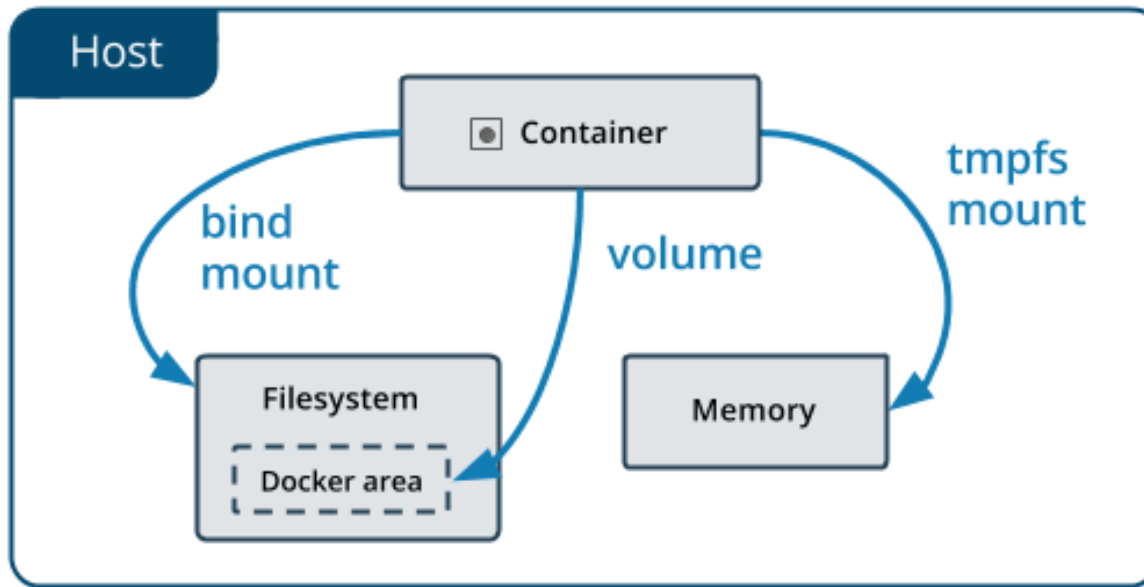
Запись на RW-уровень контейнера требует дополнительного уровня абстракции, что снижает производительность

- Рекомендованный драйвер overlay2 используется по умолчанию

Хранение данных приложения (2 из 2)

Для сохранения и совместного использования данных используются дополнительные средства:

- Тома (data volumes)
- Каталоги (bind mounts)
- Временная файловая система (tmpfs)



Итоги

В данной лекции были рассмотрены следующие темы:

-  Контейнерная виртуализация
-  Docker
-  Управление образами и контейнерами
-  Сборка образов контейнеров
-  Контейнеризация приложений
-  Подключение контейнеров к сети
-  Долговременное хранение данных